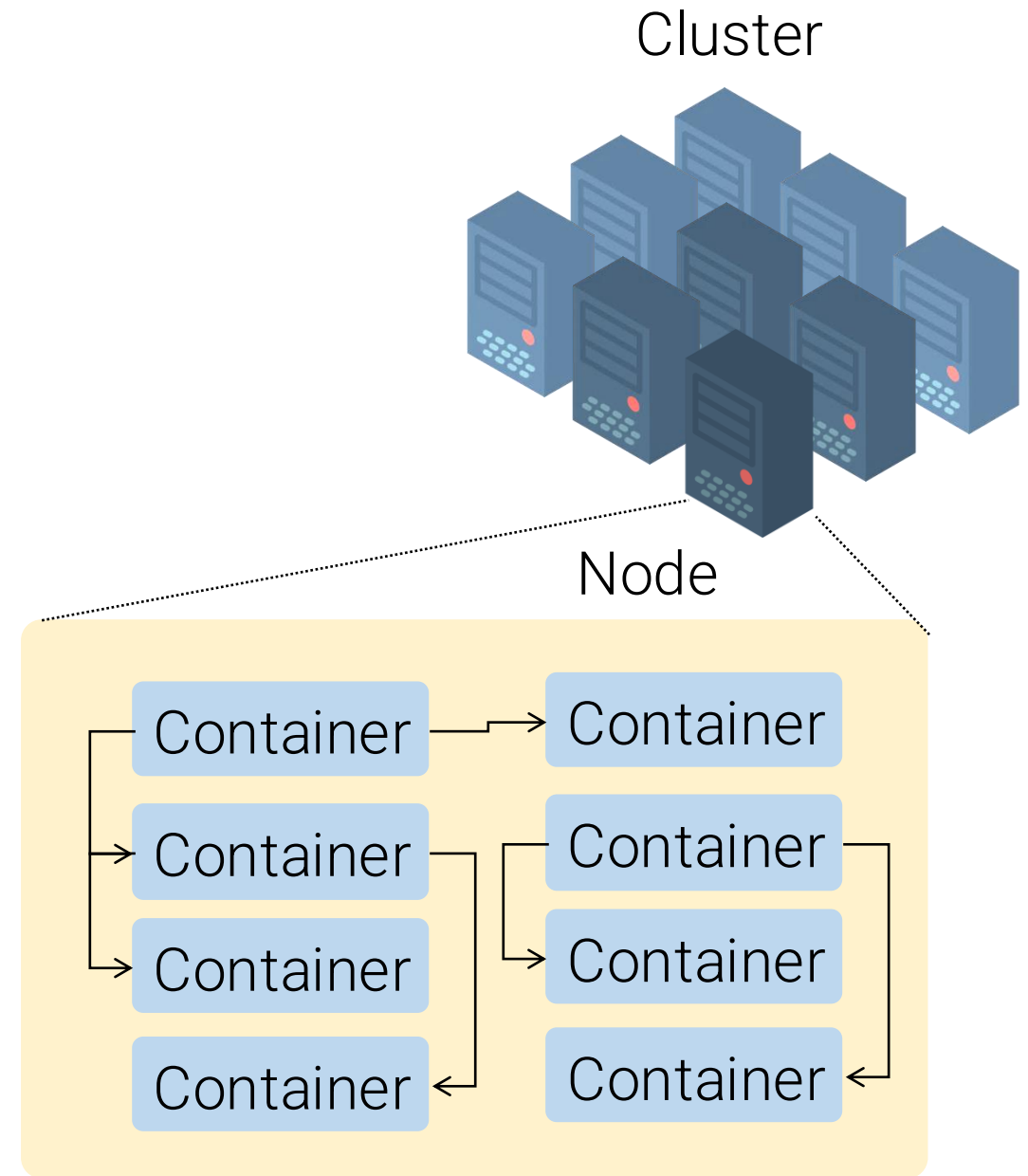


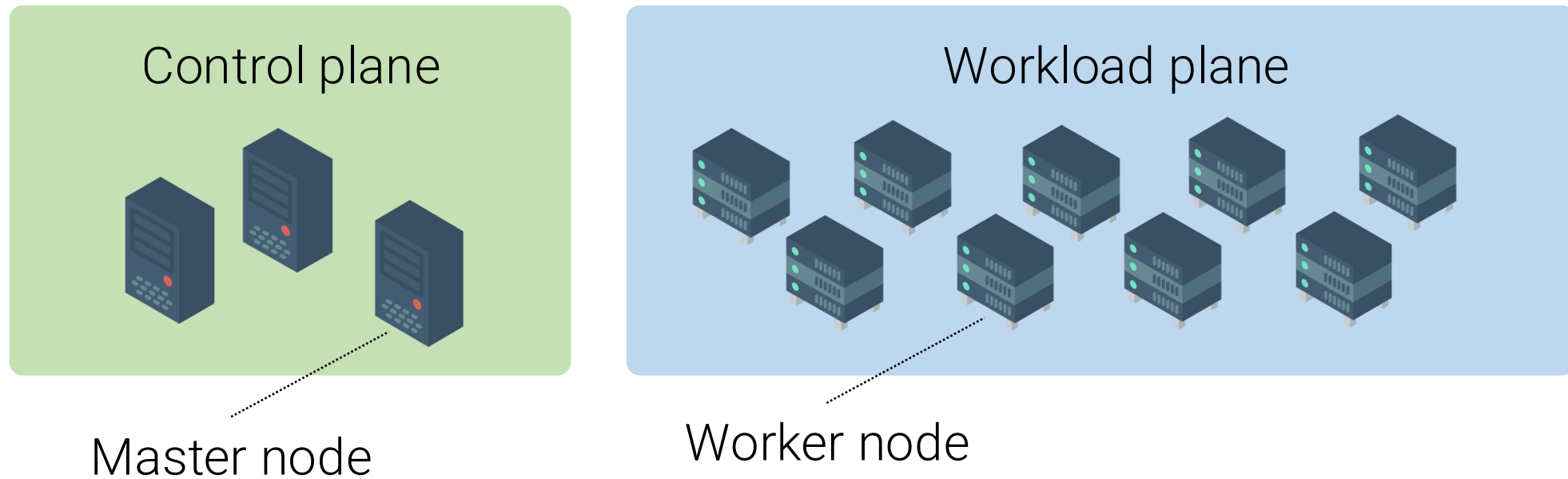
Problem

- Model as a service architecture
- Thousands of microservices deployed on multiple machines
- How to manage:
 - High availability (virtually no downtime)
 - Scalability (up and down according to demand high performance)
 - Disaster recovery (backup and restore)
 - Maintenance and software updates (apply changes consistently and smoothly to multiple nodes)



K8s architecture

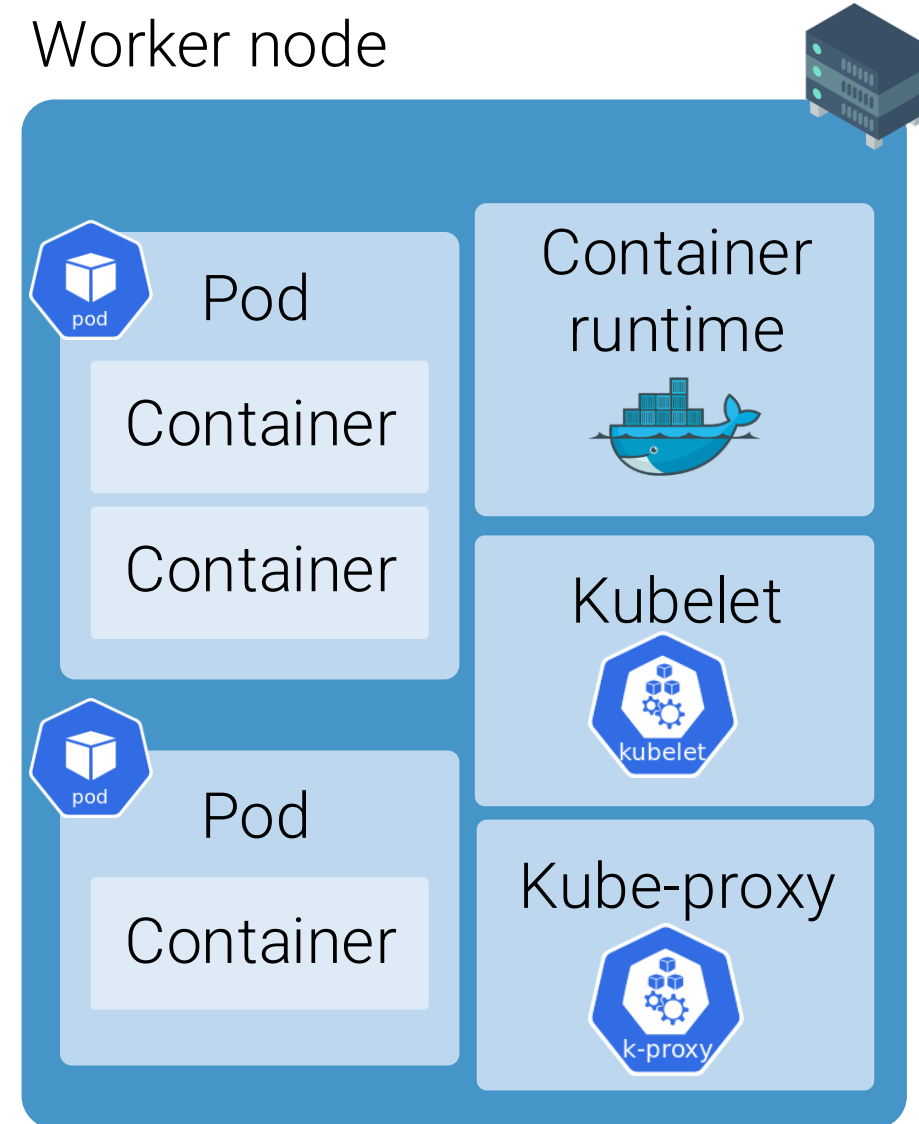
- Cluster: a set of physical or virtual nodes
- Control plane: the set of master nodes (low resources, coordination)
- Workload plane: the set of worker nodes (high resources, computation)



Worker node

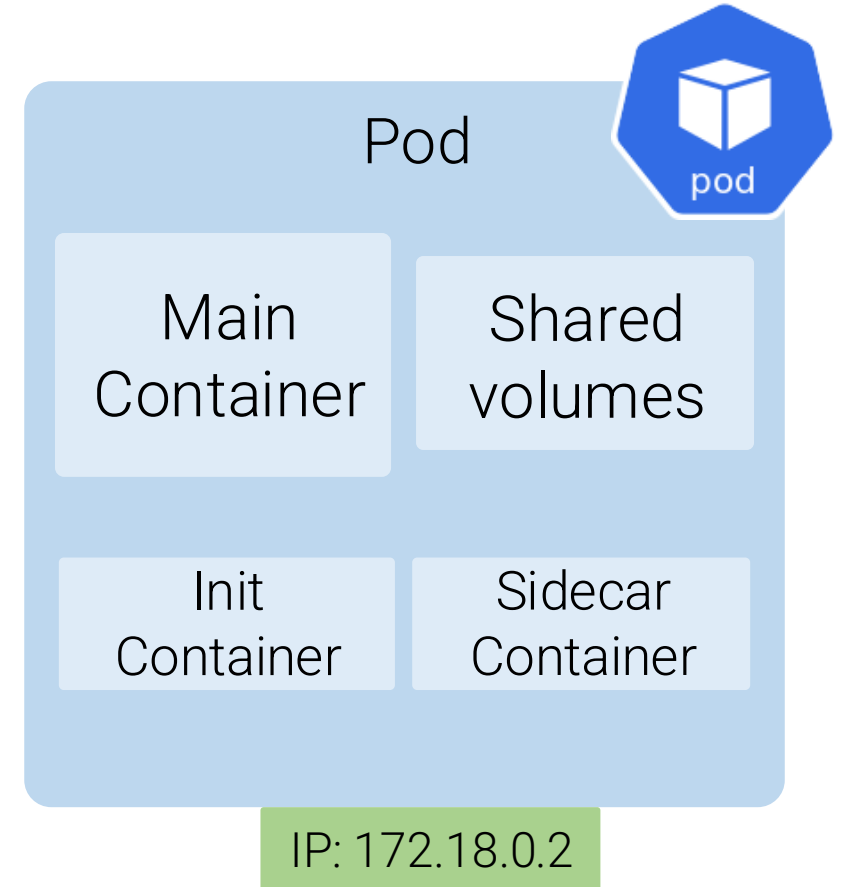
- Container runtime that runs containers
 - Docker (default), CRI-O, containerd
- Kubelet service
 - Communicate with the Master node through the API
- Kube-proxy
 - Forwards requests from services to pods

Worker node



Pods and containers

- Pod is the smallest k8s deployable unit you can configure
- Self-contained server with an IP address
- Abstraction (wrapper) of one or more containers
 - Typically, 1 container per pod, init containers and sidecar containers can be used
- 1 pod = 1 microservice
- Resides on one server only, cannot stretch across servers
- Can access shared volumes of persistent storage



Container design patterns

READ AT HOME

- Single container pattern
- Sidecar pattern
 - Extend main container functionality with a secondary container
 - Sidecar container has the same lifecycle of main container
 - Useful to extend without modifying the main container
 - E.g., web server + logging system
- Ambassador pattern
 - Helper container to send network requests on behalf of main container
 - Useful when network configurations are complex
- Init container pattern
 - Main container starts only after init container terminates
 - Several init containers can be run in sequence
 - Useful when installations or setup is needed or to delay execution of main
- Adapter container pattern
 - A sidecar container that transforms I/O to main container
 - Useful when external components that interact with container change frequently

K8s architecture concepts (summary)

- Client (your terminal) communicates to the API server through API, UI (k8s dashboard), or CLI (kubectl)
- Master is crucial for execution, at least 2 in production systems
- UI, API, CLI communicate with API server
- API server is load balanced across multiple master nodes
- Scheduler uses algorithms to determine which node is the most appropriate
- Scheduler talks with the kubelet on specific nodes to execute commands (e.g., creation of Pods)

K8s workflow - deploy

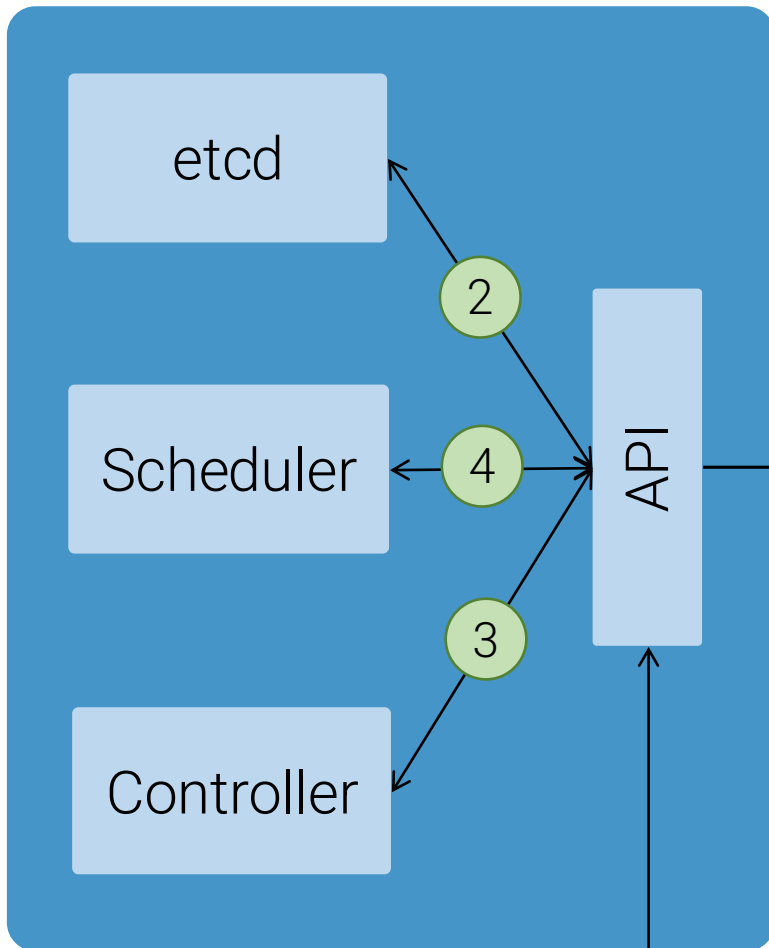
Client



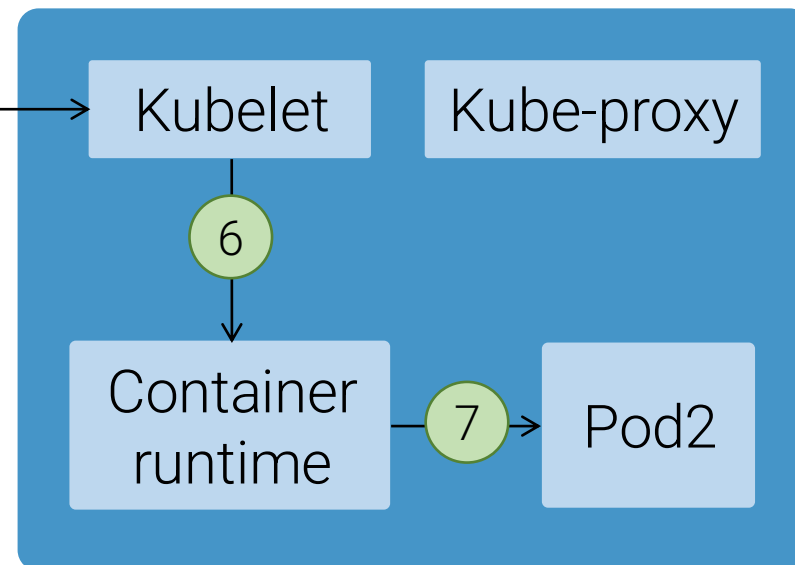
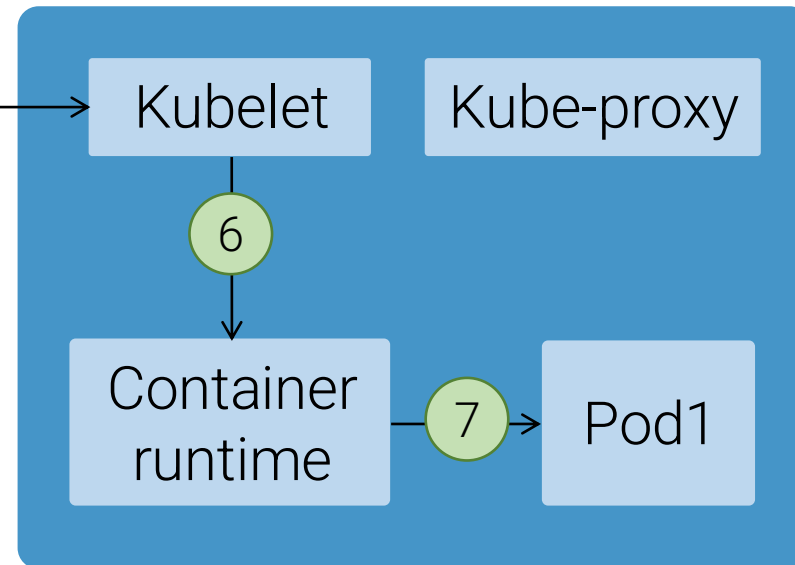
1

Deploy

Master node



Worker node 1



Worker node 2

K8s workflow – deploy (summary)

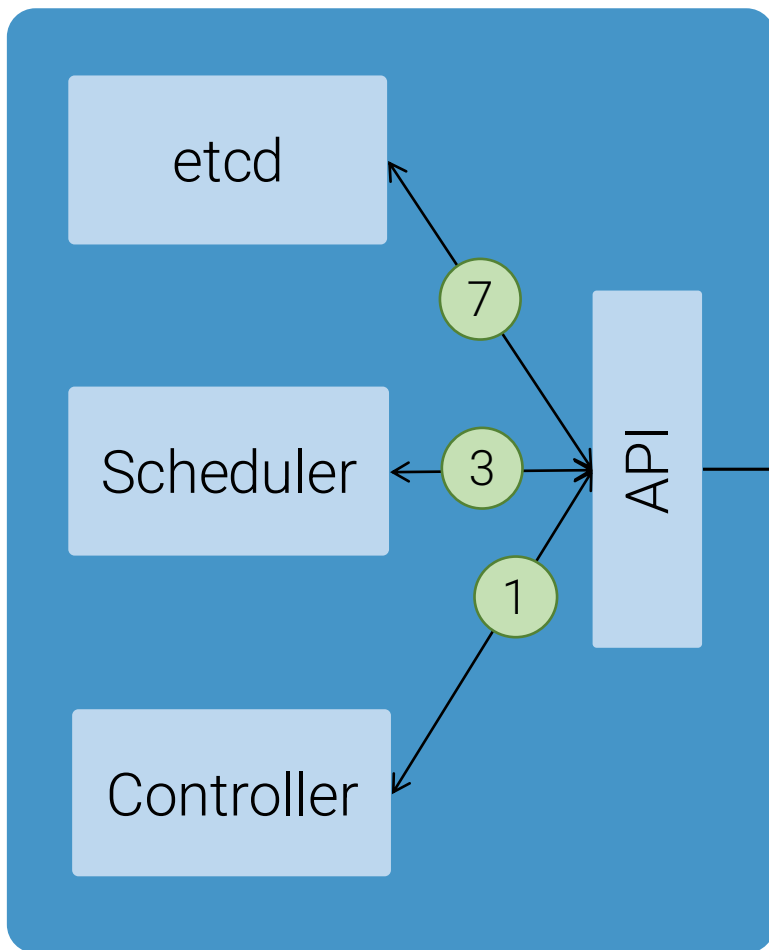
1. API server receives a valid (authenticated) deployment request
2. The deployment definition is saved in etcd
3. Controller manager is notified of the new resource
4. Scheduler is asked to allocate tasks to workers
5. Through the API, the scheduler contacts the Kubelet in the selected nodes and commands them to create new resources
6. The Kubelet asks the Container Runtime to create resources
7. New pods are created
8. Kubelets communicate back to API of successful creation
9. etcd is updated accordingly

K8s workflow - failure

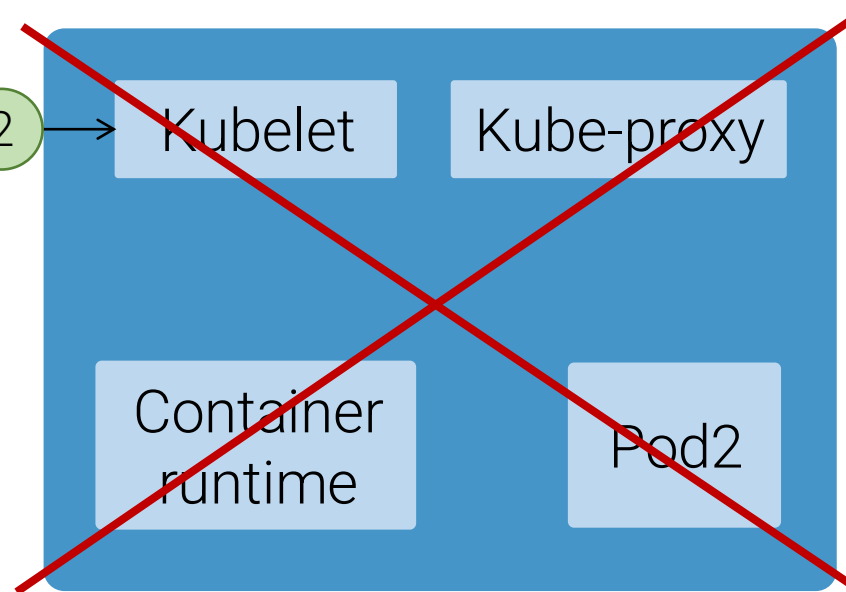
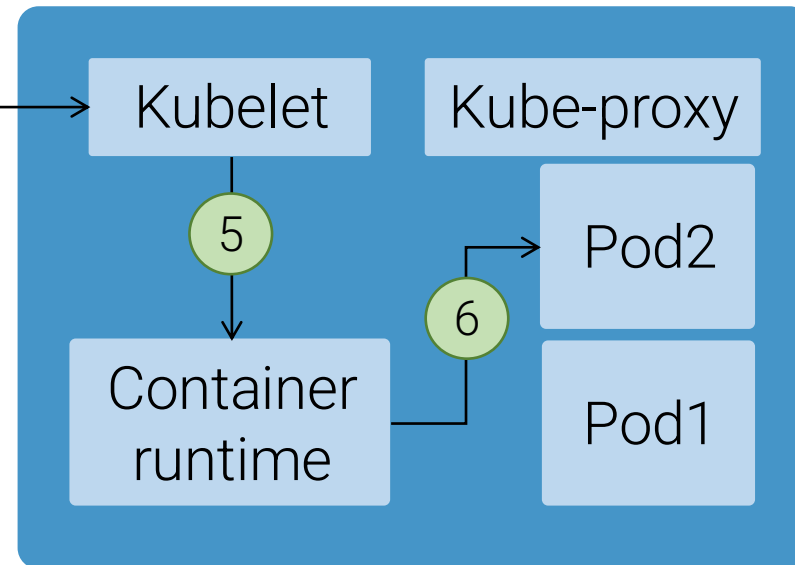
Client



Master node



Worker node 1



Worker node 2 - **FAILED**

K8s workflow – failure (summary)

1. Controller probes the worker nodes
2. Controller discovers that Pod2 is down (the whole worker is down)
3. Scheduler is asked to create re-allocate Pod2
4. Scheduler finds that Worker 1 has free resources picks it
5. The Kubelet asks the Container Runtime to create resources
6. New Pod is created
7. etcd is updated accordingly

Scheduler strategies

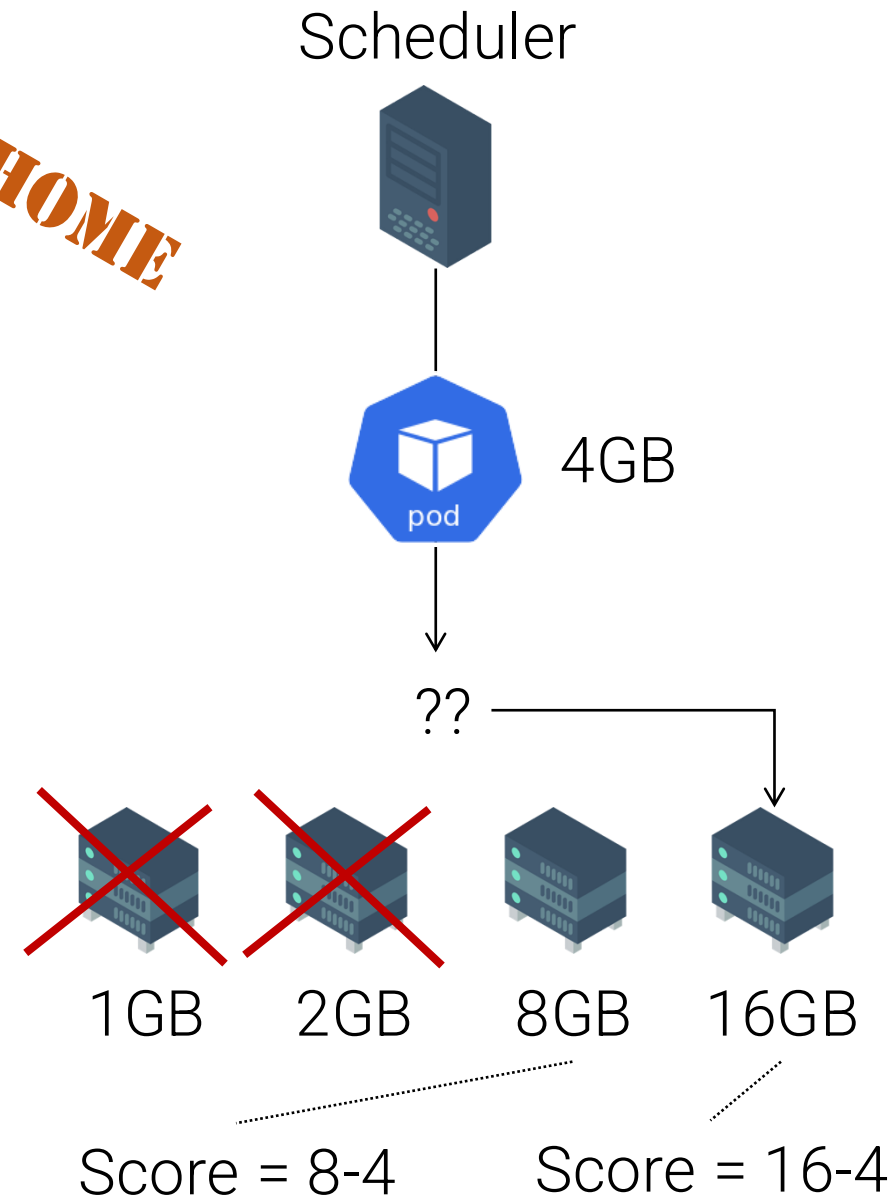
- Filtering

- Exclude nodes that don't satisfy given (customizable) predicates
- Predicate types: General, Volume, Host, Pod
- Taints and affinities

- Scoring

- Rank the remaining nodes by a priority score, according to a (customizable) set of rules

READ AT HOME

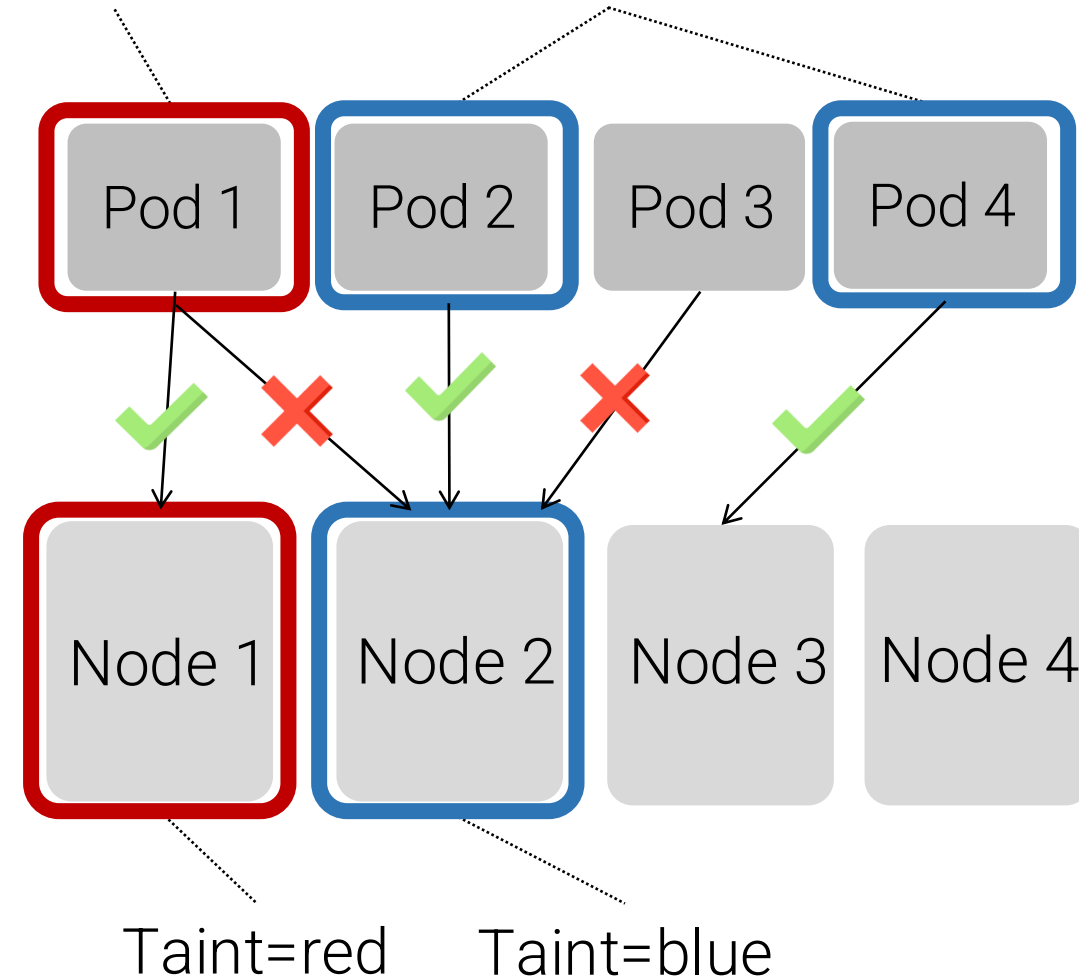


Taint & tolerations

READ AT HOME

- Nodes have a set of taints. A taint marks a Node so that the Scheduler avoids or prevents using it for certain Pods.
- Pods have tolerations for taints
- By default, worker nodes/pods have no taint/toleration
- The Taint effect is what happens to the Pod that have no toleration
 - No schedule, Prefer no schedule, No execute (relocate already running Pods)

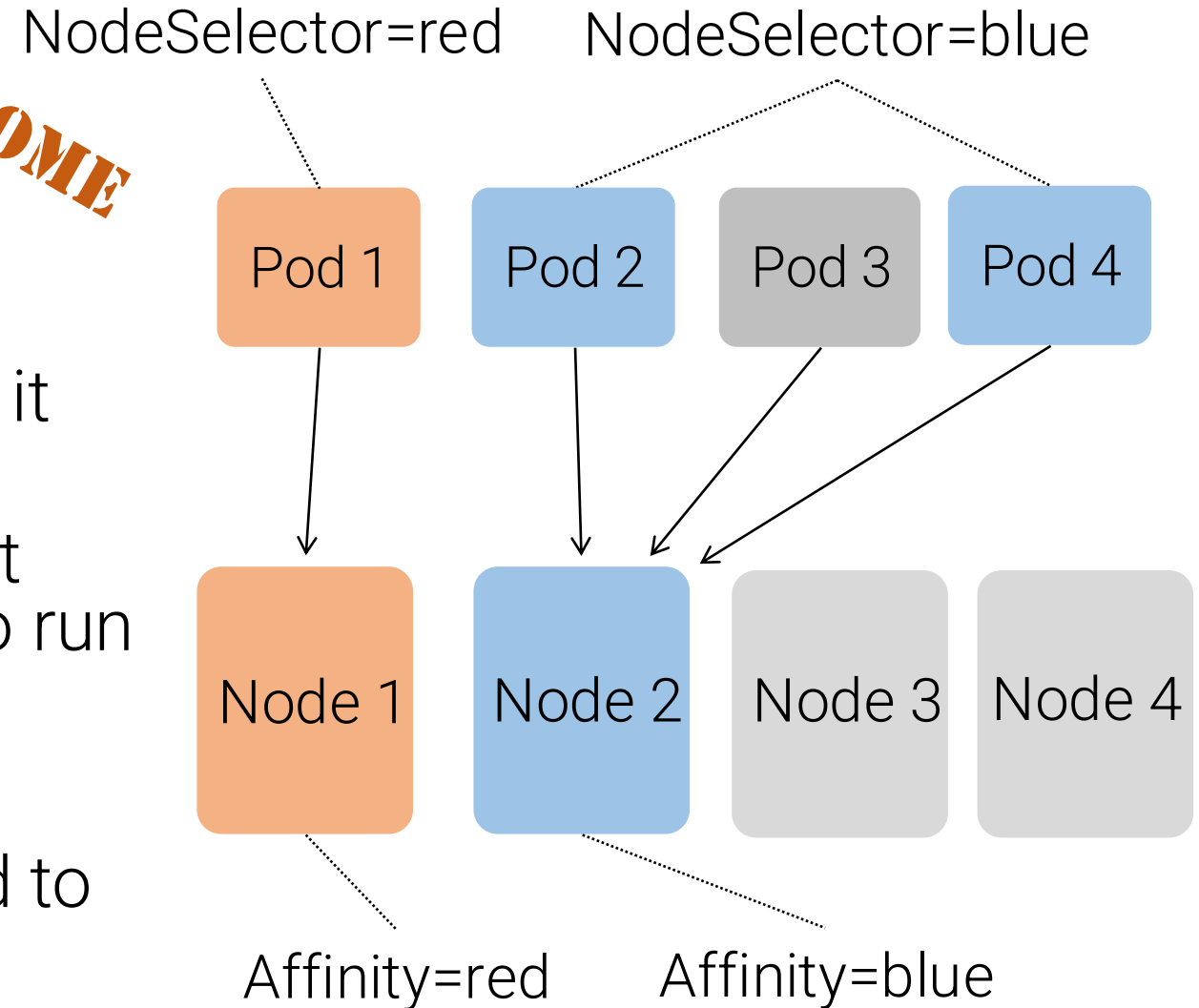
Toleration=red Toleration=blue



Affinities

READ AT HOME

- Nodes have a set of affinities
- Pods have NodeSelectors that specify the set of nodes on which it must (or should) run
- Affinity of a node does not prevent Pods without any NodeSelector to run on them
- Taints and affinities are combined to achieve nuanced effects

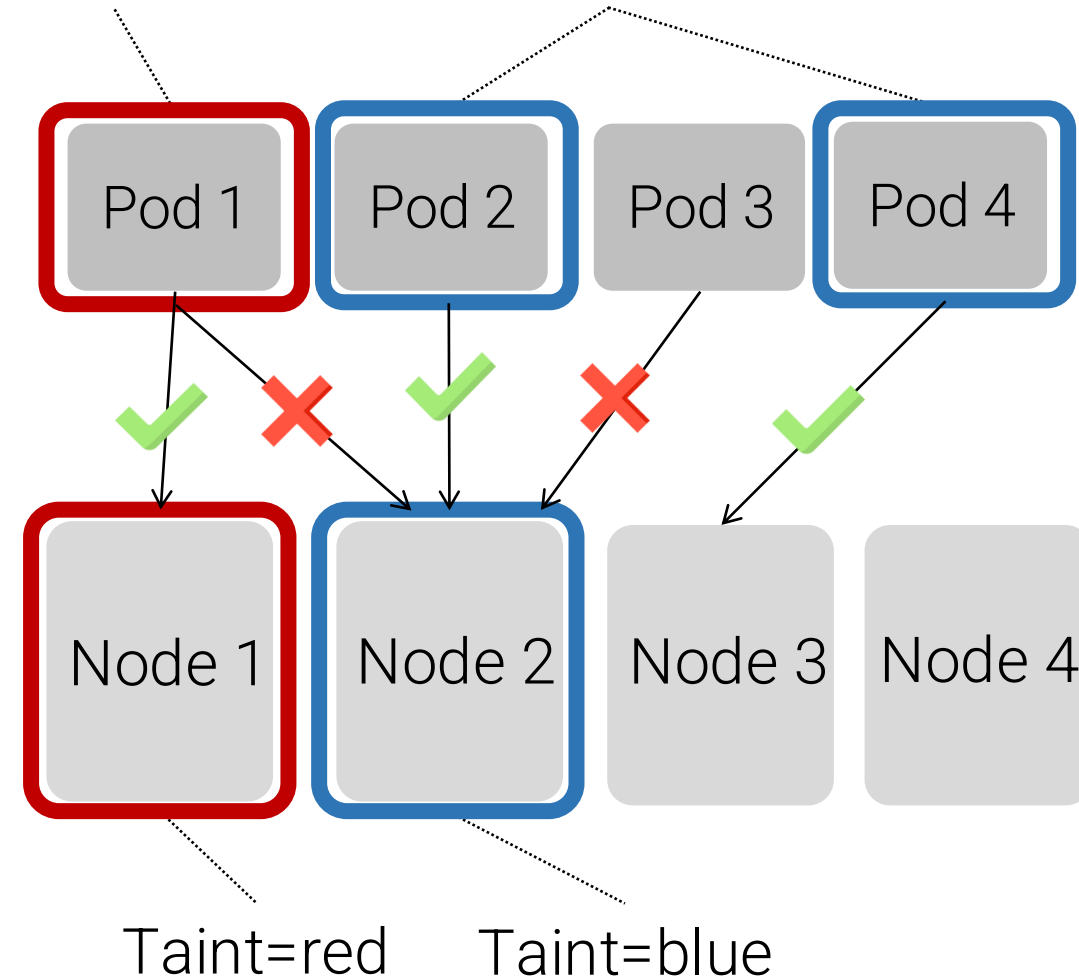


Taint & tolerations

READ AT HOME

- Nodes have a set of taints. A taint marks a Node so that the Scheduler avoids or prevents using it for certain Pods.
- Pods have tolerations for taints
- By default, worker nodes/pods have no taint/toleration
- The Taint effect is what happens to the Pod that have no toleration
 - No schedule, Prefer no schedule, No execute (relocate already running Pods)

Toleration=red Toleration=blue

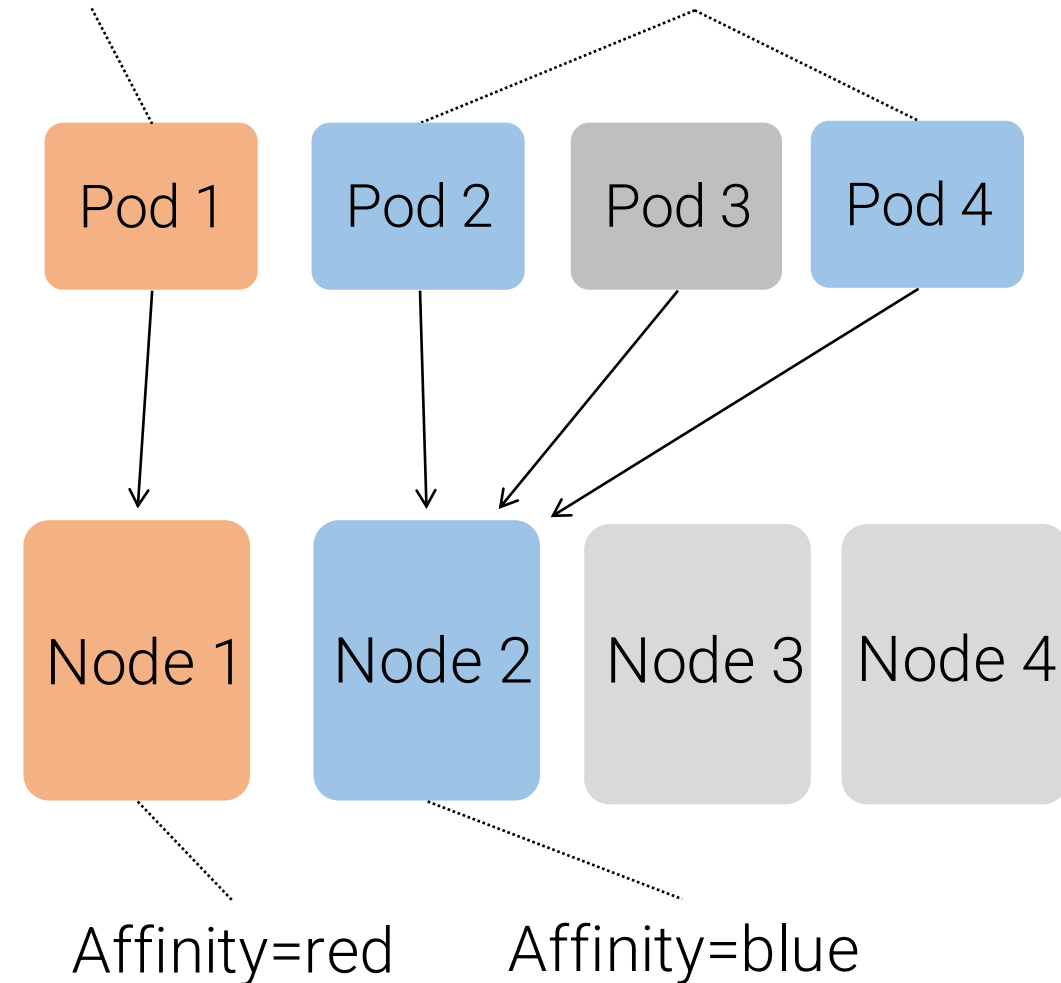


Affinities

READ AT HOME

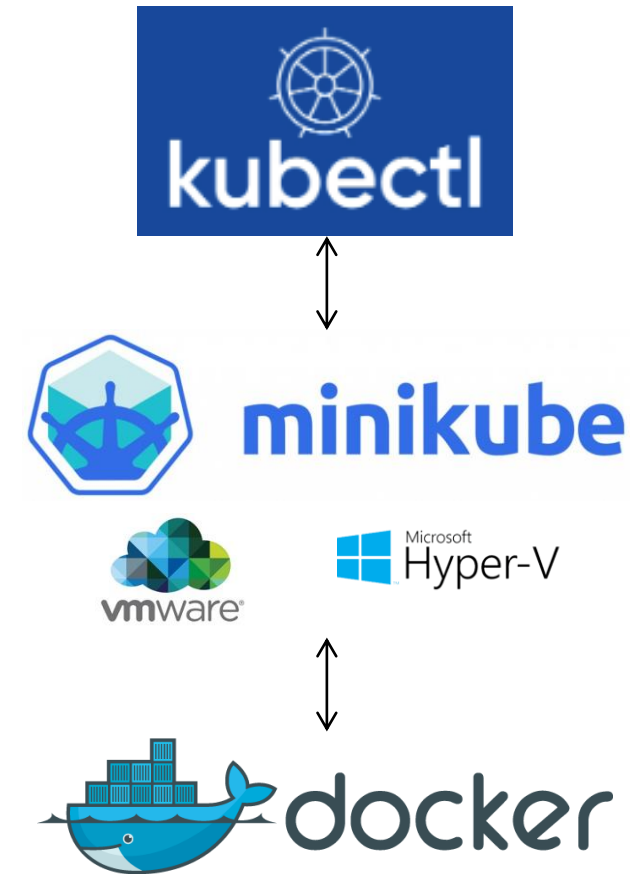
- Nodes have a set of affinities
- Pods have NodeSelectors that specify the set of nodes on which it must (or should) run
- Affinity of a node does not prevent Pods without any NodeSelector to run on them
- Taints and affinities are combined to achieve nuanced effects

NodeSelector=red NodeSelector=blue



Local k8s setup (summary)

- Minikube
 - Create virtual machine on your laptop that simulates a Kubernetes cluster
 - The virtual machine will host both master and workers
- Container runtime
 - Docker (default choice for k8s)
 - Included in the minikube distribution
- CLI
 - Kubectl, a command line tool to interact with the API server
 - Included in the minikube distribution
 - Minikube has its own CLI, but we will use it only for cluster creation

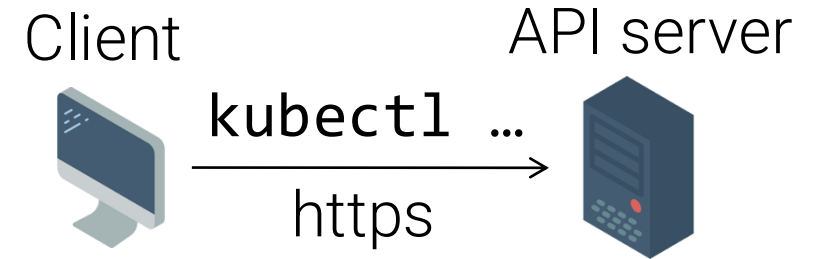


Minikube

```
type: Control Plane  
host: Running  
kubelet: Running  
apiserver: Running  
kubeconfig: Configured
```

- Use minikube for starting, stopping, and deleting cluster
 - `minikube` (for list of commands)
 - `minikube start --driver=[hyperkit | hyperv | virtualbox]`
 - `minikube status`
- `kubectl cluster-info` shows the IP address assigned to the VM running the k8s node
- Run `docker ps` on the node to see running containers different that implement different services (e.g., apiserver)

Kubectl



- Pronounced kube-C-T-L, kubecontrol, kubecat1
- Command line tool for k8s cluster to connect to cluster and manage it remotely
- Deploy, inspect, edit resources, view logs, debug, etc.

```
kubectl [command] [TYPE] [NAME] flags
```

Run a “Naked” Pod and monitor status

Get information about the nodes

```
kubectl get nodes
```

Get information about running system Pods on master node

```
kubectl get pods
```

Run a new Pod based on a Dockerhub image

```
kubectl run containername --image=imagename  
kubectl run mywebserver --image=nginx
```

Get information about container

```
kubectl describe pod containername
```

Pods are almost never created directly – **deployments** are used

Run a Pod through Deployment

You won't need to
take care of anything
below this level

Create a new nginx Deployment using a Dockerhub image

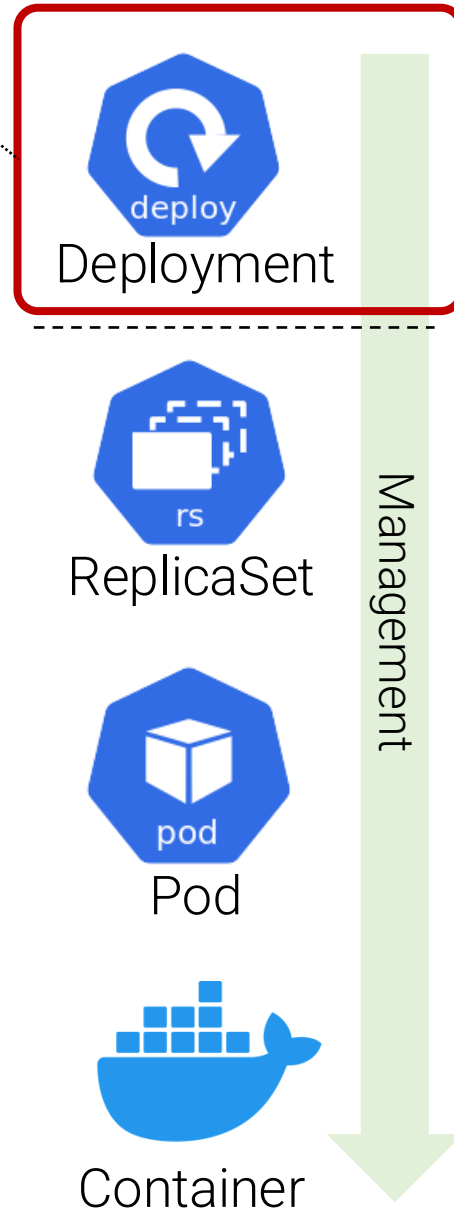
```
kubectl create deployment mynginx-deploy --image=nginx
```

Check deployment information

```
kubectl describe deployment mynginx-deploy
```

Automatically generated configuration file:

```
Name: mynginx-deploy
Selector: app=mynginx-deploy
Replicas: 1 desired | ... | 1 total
NewReplicaSet: mynginx-deploy-688b86db96 (1/1)
Pod Template:
  Labels:  app=mynginx-deploy
  Containers:
    nginx:
      image: nginx
```



Some Deployment commands

Get list of deployments

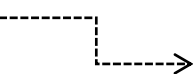
```
kubectl get deployments
```

List pods to see their name are prefixed with the deployment name

```
kubectl get pods
```

See references to ReplicaSet and Deployment

```
kubectl describe pod podname
```



```
Name: mynginx-deploy-688b...  
Controlled By: ReplicaSet/mynginx-deploy-688b...
```

Scale up a deployment to 5 replicas

```
kubectl scale deployment deploymentname --replicas=5
```

Delete the deployment

```
kubectl delete deployment deploymentname
```

For debugging

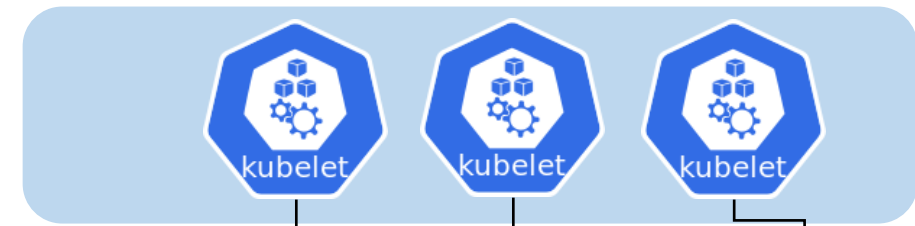
```
kubectl logs podname # print pod logs
```

```
kubectl exec --stdin --tty podname -- /bin/bash # enter pod bash
```

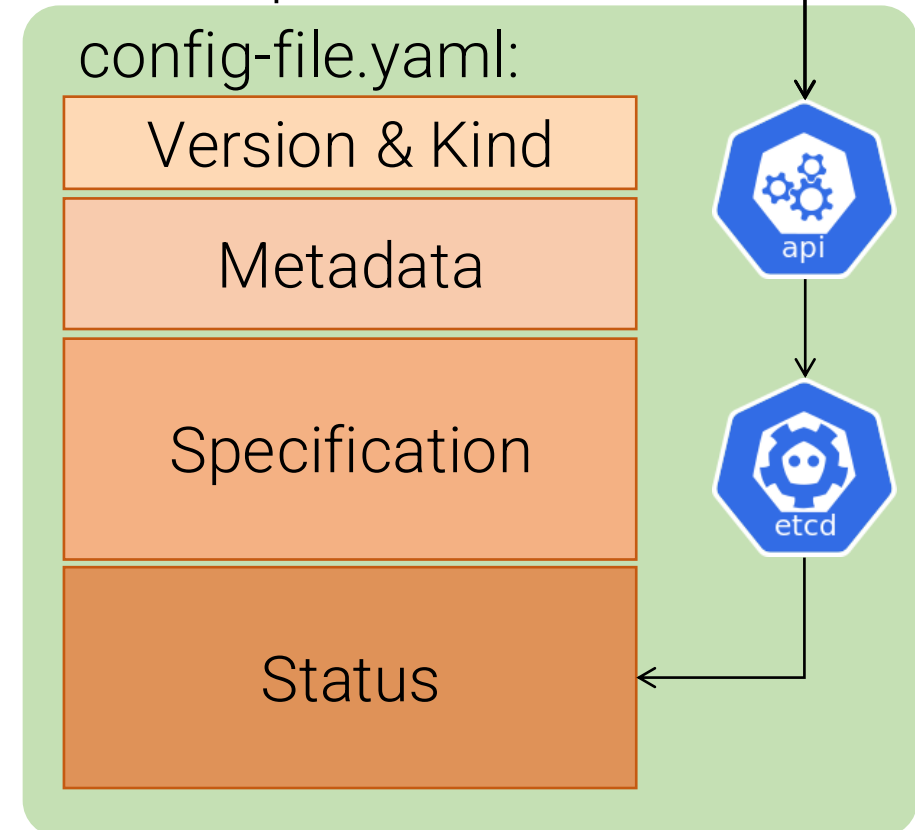
K8s configuration file structure

- Header
 - Config file version and kind (deployment, service)
- Metadata
- Specification
 - The desired state of the system
- Status
 - Automatically generated and maintained by Kubernetes
 - Information is pulled from etcd, which in turn is updated by the API based on the information gathered from the kubelets on the nodes
- K8s self-healing is achieved by comparing the status and the specification and taking actions to fill the gap between the two
- Best practice: store config file with code (similar to docker-compose)

Workload plane



Control plane



Application deployment example

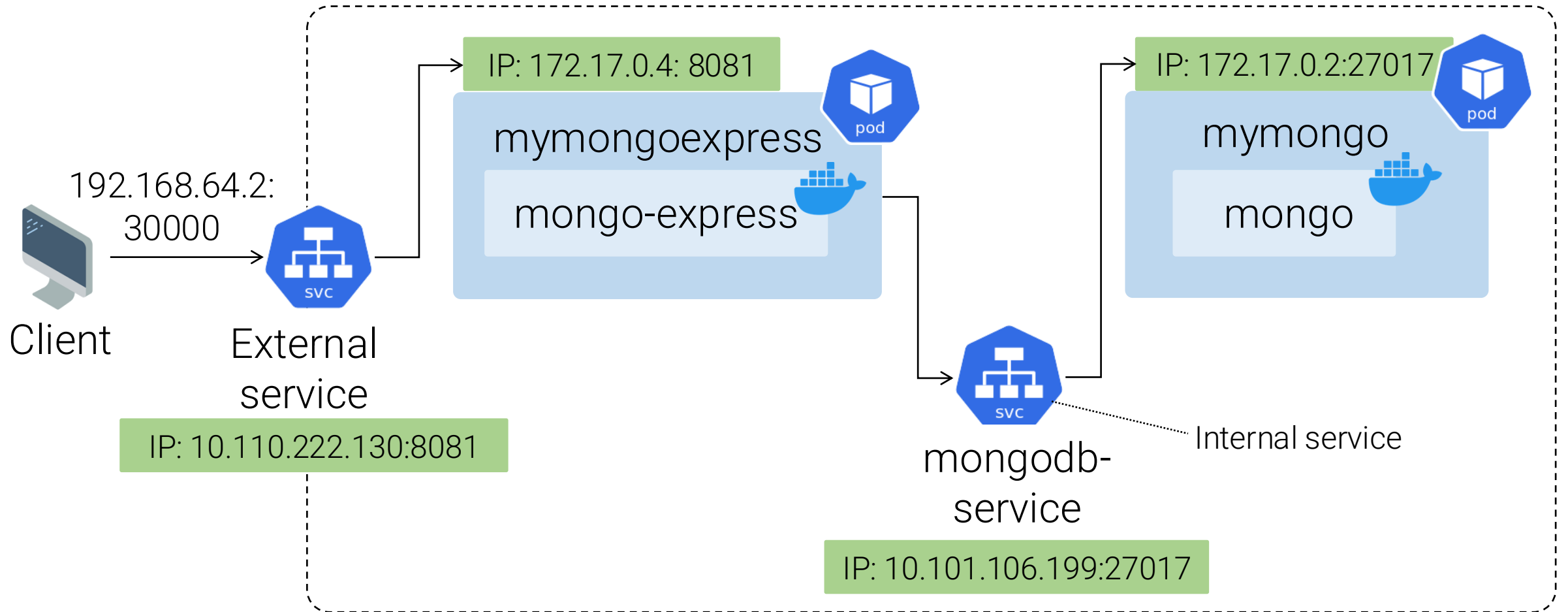
Config files:

mongoexpress.yaml

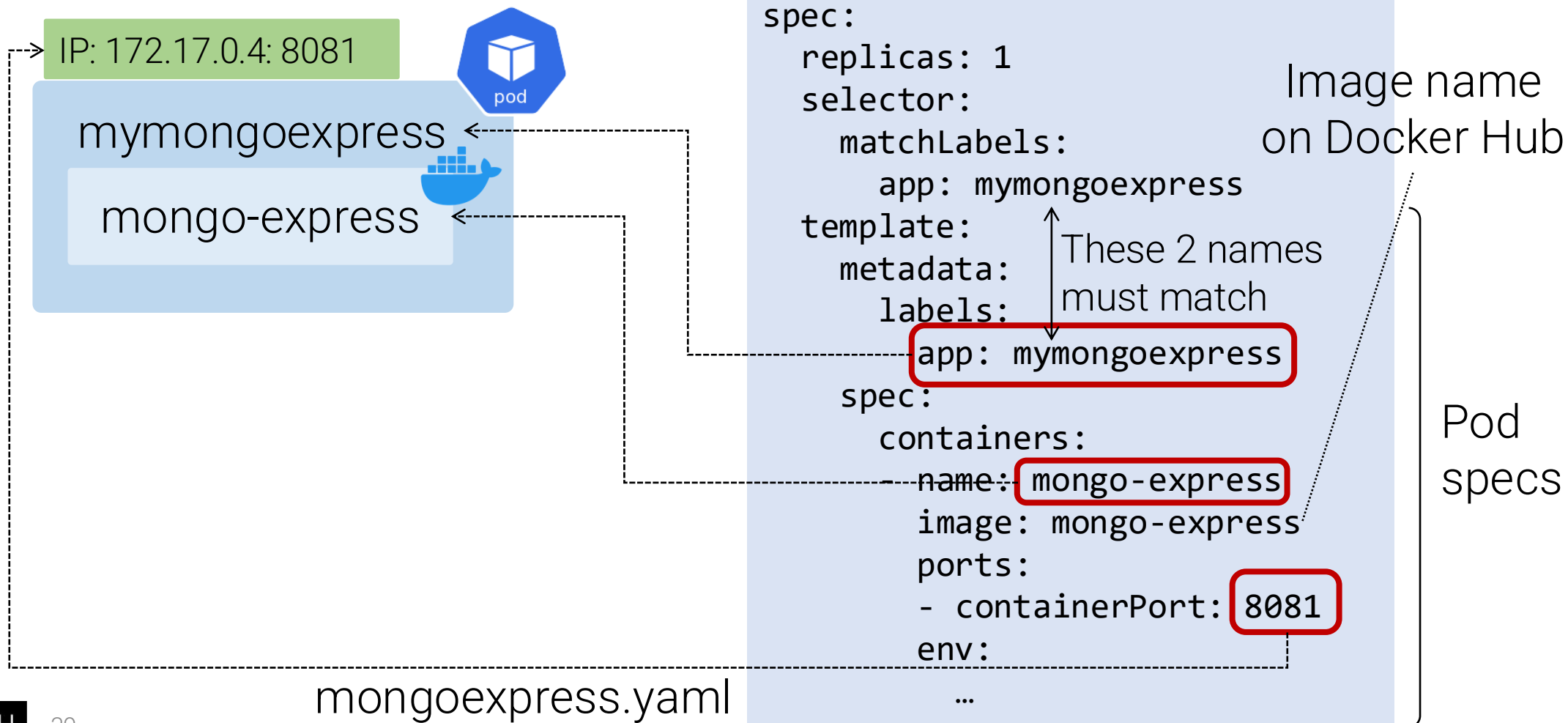
configmap.yaml

secret.yaml

mongo.yaml



Mongoexpress deployment



Mongoexpress deployment

IP: 172.18.0.1: 8081



mymongoexpress
mongo-express



Check on
DockerHub for
the env variables
to be set

https://hub.docker.com/_/mongo-express

mongoexpress.yaml (cont'd)

secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: mongodb-secret
type: Opaque
data:
  mongo-root-username: dXN1cm5hbnU=
  mongo-root-password: cGFzc3dvcmQ=
```

```
env:
  - name: ME_CONFIG_MONGODB_ADMINUSERNAME
    valueFrom:
      secretKeyRef:
        name: mongodb-secret
        key: mongo-root-username
  - name: ME_CONFIG_MONGODB_ADMINPASSWORD
    valueFrom:
      secretKeyRef:
        name: mongodb-secret
        key: mongo-root-password
  - name: ME_CONFIG_MONGODB_SERVER
    valueFrom:
      configMapKeyRef:
        name: mongodb-configmap
        key: database_url
```

echo -n 'password' | base64

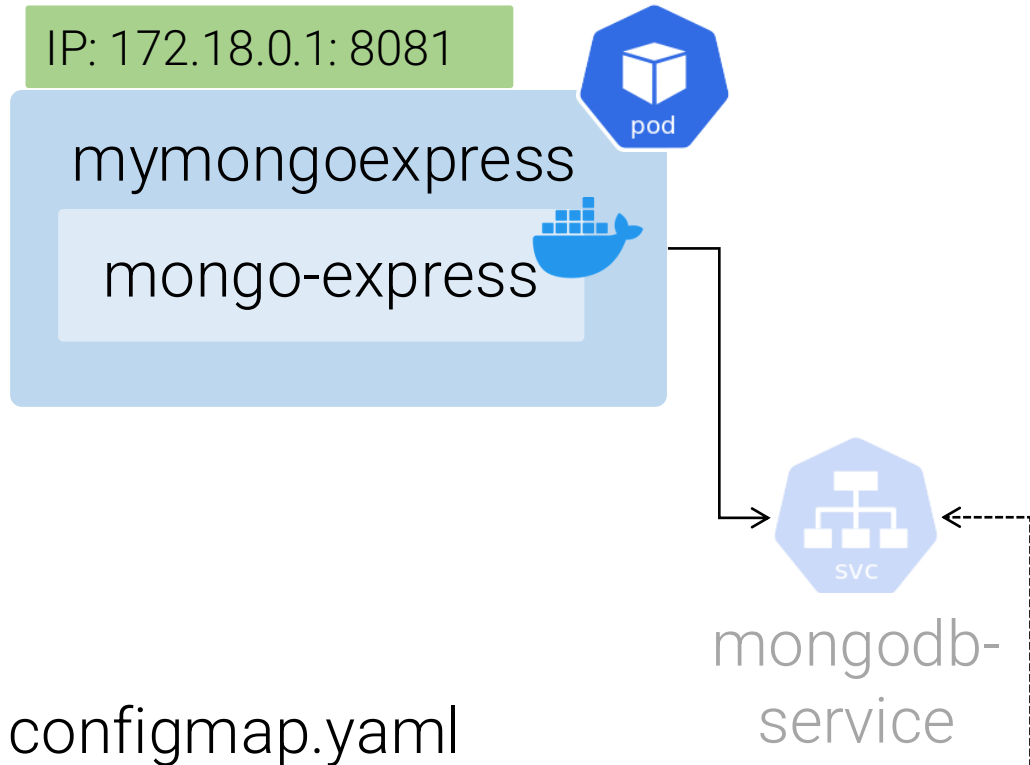
Create base64 username/passwd

Secret concepts (summary)

- Information in secret configurations does not leave the node
- Only a minimal security measure. Encryption, certificates, etc. can be set up
- Values in the secret configuration must be base-64 encoded
- Values in the secret configuration are accessed from the deployment using the name of configuration and the key from the desired key,value pair

Mongoexpress deployment

mongoexpress.yaml (cont'd)



configmap.yaml

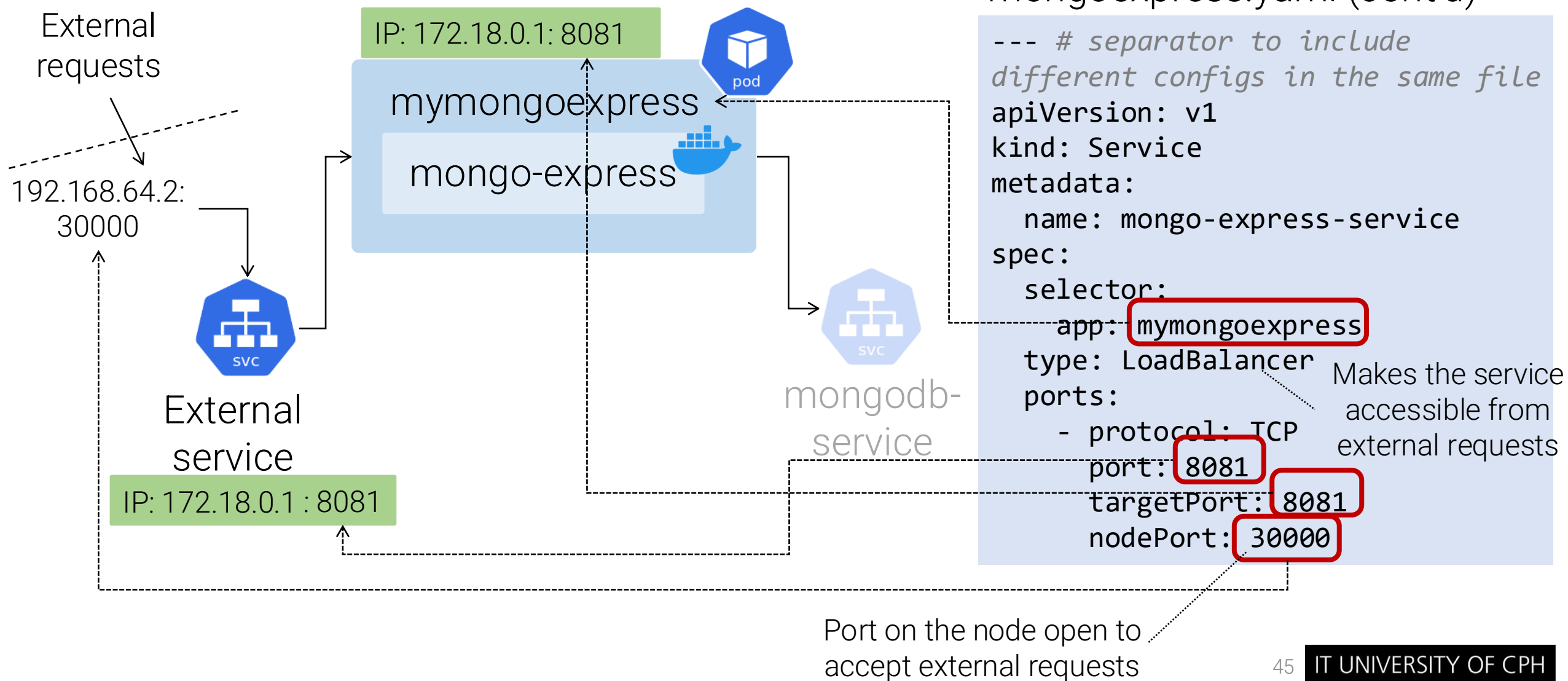
```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mongodb-configmap
data:
  database url: mongodb-service
```

```
env:
- name: ME_CONFIG_MONGODB_ADMINUSERNAME
  valueFrom:
    secretKeyRef:
      name: mongodb-secret
      key: mongo-root-username
- name: ME_CONFIG_MONGODB_ADMINPASSWORD
  valueFrom:
    secretKeyRef:
      name: mongodb-secret
      key: mongo-root-password
- name: ME_CONFIG_MONGODB_SERVER
  valueFrom:
    configMapKeyRef:
      name: mongodb-configmap
      key: database url
```

ConfigMaps concepts (summary)

- Used to store non-confidential key-value pairs
- The data in ConfigMaps can be shared across multiple deployment, so that changing the values propagates to all deployments
- Values in the ConfigMaps are accessed from the deployment using the name of ConfigMap and the key from the desired key,value pair

Mongoexpress service



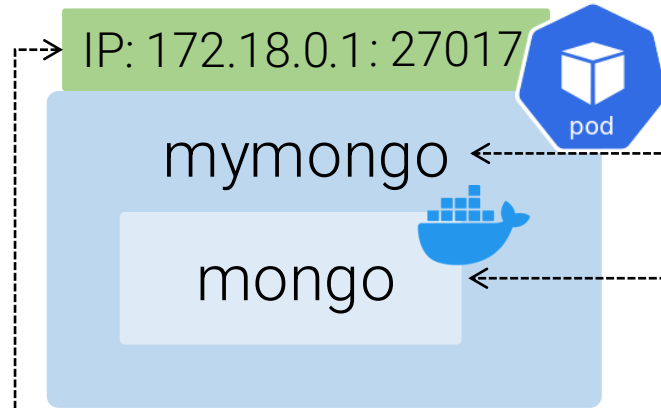
Service concepts (summary)

- Life-cycle decoupled from the Pods
- Stable point of connections between Pods
- Can have their own configuration file but usually they refer to a specific deployment, so it makes sense to write one single file with deployment + service specification
- Services can be exposed to an external IP address, with a port in the range [30000-32767]

Mongo deployment

header

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mymongo
  labels:
    app: mymongo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mymongo
  template:
    metadata:
      labels:
        app: mymongo
    spec:
      containers:
        - name: mongodh
          image: mongo
          ports:
            - containerPort: 27017
          env:
```



mongo.yaml

Mongo deployment

mongo.yaml (cont'd)

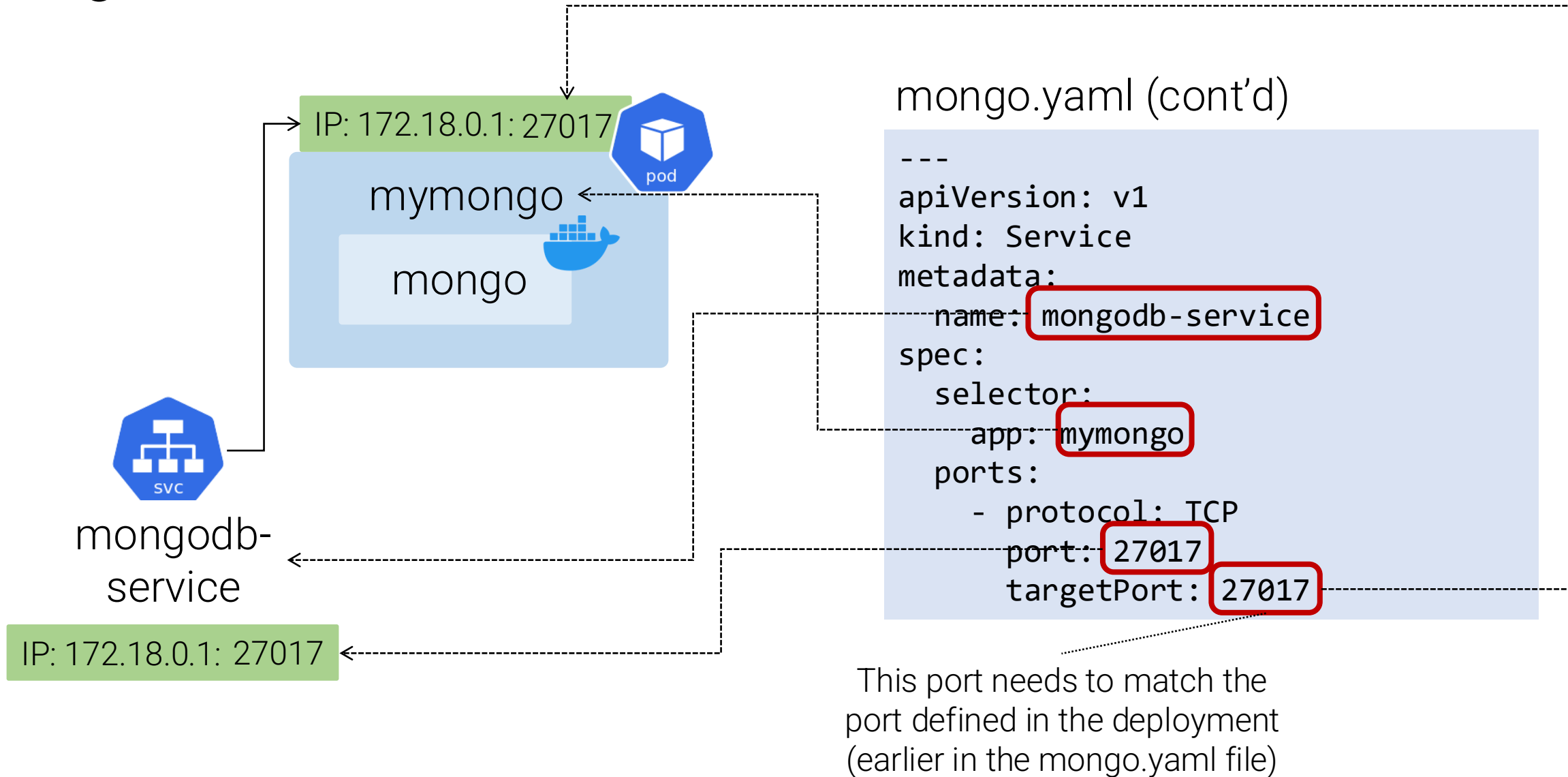


secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: mongodb-secret
type: Opaque
data:
  mongo-root-username: dXN1cm5hbWU=
  mongo-root-password: cGFzc3dvcmQ=
```

```
env:
  - name: MONGO_INITDB_ROOT_USERNAME
    valueFrom:
      secretKeyRef:
        name: mongodb-secret
        key: mongo-root-username
  - name: MONGO_INITDB_ROOT_PASSWORD
    valueFrom:
      secretKeyRef:
        name: mongodb-secret
        key: mongo-root-password
```

Mongo service



Deployment commands

```
kubectl apply -f secret.yaml # apply or create the deployment
kubectl apply -f mongo.yaml # apply secret and configmap first (dependencies)
kubectl apply -f . # apply all yaml files in current dir

kubectl get [all | service | deployment | pod ...] -o wide # show component info
(e.g., IP)
kubectl describe service servicename # shows selector, endpoints
kubectl get deployment deploymentname -o yaml > fileout # see the
automatically-generated status at the bottom of the config file

kubectl delete -f mongo.yaml # config files are used to delete too

# clean up state
kubectl delete deployments --all
kubectl delete services --all
kubectl delete pods --all
kubectl delete daemonset --all
```

How to deploy our application

```
kubectl apply -f secret.yaml # apply all configurations
kubectl apply -f configmap.yaml
kubectl apply -f mongo.yaml
kubectl apply -f mongoexpress.yaml
kubectl apply -f . # in alternative, apply all yaml files in the current dir
kubectl get all # visualize running services, pods, etc.
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP
mongo-express-service	LoadBalancer	10.110.222.130	<pending>	8081:30000/TCP
mongodb-service	ClusterIP	10.101.106.199	<none>	27017/TCP

```
minikube service --url mongo-express-service # get external URL (minikube only)
```

```
http://192.168.64.2:30000/ # use this URL to access the application
```

```
kubectl delete all --all # deletes all the deployments, services, etc.
```

<https://minikube.sigs.k8s.io/docs/handbook/accessing/>

Scale it up!

- The configuration files can be modified and redeployed
- K8s will make sure to realign the current status to the new desired status

```
# Option 1: change deployment YAML file and rerun apply  
kubectl apply -f mongoexpress.yaml
```

```
# Option 2: command line argument  
kubectl scale deployments/deploymentname --replicas=N
```

Exercise

- Run the (mongo + mongo-express) deployment explained in class
- Create k8s deployment for sentiment analysis exercise (last week)
- Create k8s deployment for RecSys exercise (last week)
 - Modify exercise such that
 - Volumes are not used
 - The User simulator Python code refers to the right k8s service names defined in deployment .yaml files
 - The feedback model print in stdout a message every time it receives a tuple
 - Push the images of the three modules on Docker Hub
 - Inspect logs of the pod running the feedback module to make sure recommendation errors are logged
 - Scale the RecSys module up (from 1 node to 2 nodes)